Jihwan Choi

Seoul | jhwan333@gmail.com | 010-9692-6526 | jihwan-choi | jhwanchoi | jhwanchoi.github.io

Profile

Software engineer building and operating **production Generative AI** and **MLOps inference/evaluation platforms** for autonomous-driving AI. I build a **cost-aware multimodal VLM evaluation system** (rules → local VLM → AWS Bedrock escalation), **LLM cost/usage observability**, **RAG**, and **agentic tooling**. I re-architected a shared Kubernetes GPU cluster for GitOps operations after taking it over, **led scheduler and queue policy design**, and **own the design, implementation, and operation of SVEvalFlow inference/evaluation workflows**. MTBF completed **430,851 processing commands with 99.87% pipeline processing success** as of June 2026. GPU benchmarking and quote-based cost analysis supported an on-premise transition with an estimated **55% cost savings versus AWS**. A log-analysis platform processing **80M daily events prevented approximately KRW 100M/month in overbilling**.

Experience

Stradvision, Software Engineer

Seoul, South Korea
July 2022 – Present

Shared GPU Cluster & Evaluation Workflows [Link]

SVEvalFlow design, implementation, and operation (Oct 2025 – Present)

Tech: RKE2, Ray, KubeRay, KServe, Knative, MLflow, Airflow, ArgoCD, Harbor, lakeFS, GitHub Actions, Helm

- Took over a shared RKE2 Kubernetes cluster (60+ heterogeneous GPUs) and re-architected it for GitOps operations, **leading scheduler & queue policy design**. Shipped an application-level scheduler that preempts only the GPU deficit from running bulk jobs for urgent evaluations and automatically restores capacity, while integrating 15+ MLOps components (Ray/KubeRay, MLflow, Airflow, ArgoCD, KServe, Harbor, etc.)
- Collaborate on the training workflow (SVDeepFlow) and **solely own the inference/evaluation workflow (SVEvalFlow)**—composed of Inferit, MTBF, and PRISM below
- Built 3 CI/CD pipelines (GitHub Actions + ArgoCD) and Scale-to-Zero serverless inference (KServe/Knative) to cut idle GPU cost
- Drove internal adoption of AI-native development tooling (Claude Code, Gemini CLI, Codex; MCP/Plugin/Skill) through hands-on engineering sessions

Model Execution Image Management (Inferit) [Link]

Execution environments and artifacts for the svnet3 perception model

Tech: FastAPI, Airflow, Kubernetes, lakeFS, Docker, PostgreSQL, HCP Object Storage

- Built version/config/artifact management for svnet3 (in-house autonomous-driving vision model), and integrated an MTBF API that resolves or builds worker images by branch, config, commit, and CUDA toolchain (cu118/cu128), tracking the latest base, version, digest, and custom overrides
- Versioned customer option files (e.g., camera parameters), **removing the 1–2 days of engineer-PM search and hand-off** each ticket previously required
- Supported **5+ major release versions** and customer-ticket-driven automated inference, integrated with evaluation automation

Experience (continued)

Stradvision, Software Engineer — Jul 2022 – Present

GPU Inference & Validation (MTBF) [Link]

Autonomous-driving perception models (Mar 2025 – Present)

Tech: FastAPI, Airflow, Kubernetes (RKE2, NKS), Docker, PostgreSQL, HCP/NCP Object Storage, AWS Bedrock, Prometheus, Grafana, Loki, DCGM, Keycloak/Okta

- Progressively modernized the initial system, connecting GPU inference for the svnet3 perception model, data validation, evaluation results/history, and a Visual Inspection UI using FastAPI, Airflow, and Kubernetes
- Completed **430,851 processing commands with 99.87% pipeline processing success** (cumulative, Jun 2026). Achieved **205% higher pipeline throughput and 67.3% lower processing time** versus the previous infrastructure (2025)
- Built and operated seven types of data-validation pipelines and real-time dashboards. Automated workflows supporting pre-/post-release review of false-positive and automatic emergency braking (AEB) flags, enabling **single-person operation** of large-scale validation. Applied review assistance using rules, local VLMs, and AWS Bedrock, with only human-approved results entering final KPIs
- Proposed and executed a cloud-to-on-premise transition, including GPU hardware benchmarks, specification proposals, and quote-based cost analysis, achieving an estimated **55% cost savings versus AWS**
- Migrated on-premise MTBF to **NCP NKS/L40S, Object Storage, and NCR, completing the full cutover in Sep 2026**. Migrated PostgreSQL history and storage; checked migration completeness against object listings and batch manifests
- Built **Prometheus/Grafana/Loki/DCGM monitoring and validated GPU metrics**. Implemented **Keycloak/Okta SSO integration** for development and production environments
- Processed **57,745 commands for 1,068 hours of driving data using 56 L40S GPUs in 23h 57m**, with **57,710 completions (99.94%)**. Recorded **no VRAM incidents or pod evictions in that run**
- Validated **GPU node autoscaling from 1 to 14 within 13 minutes**, returning to 1 after completion of that NCP run

Image Processing & Resimulation (PRISM) [Link]

Jira-integrated ISP conversion and version-specific inference (Nov 2025 – Present)

Tech: FastAPI, Airflow, PostgreSQL, AWS (SQS, Lambda), Terraform

- **Phase 1 (pre-deployment)**: Implemented Jira Webhook → SQS → Lambda integration for image signal processing (ISP) conversion and Terraform configuration. Connectivity testing completed; full deployment is planned after the Inferit rollout
- **Phase 2 (resimulation, in development)**: Built the initial implementation and support another team's developer. Developing version-specific inference integration using resimulation Docker images and Airflow

Experience (continued)

Stradvision, Software Engineer — Jul 2022 – Present

Data & Labeling Automation *(2022 – 2025)*

- **WLS:** Analyzed 80M daily labeling-log events to prevent approximately KRW 100M per month in overbilling (settlement-audit confirmed). Pinpointed bottlenecks feeding UI/UX and the ALAS model (+20% labeling efficiency) (Django, MongoDB, AWS SQS)
- **ALAS:** Auto-labeling microservice, reduced manual annotation workload by ~35%; ECS-to-EKS migration and CDK-based IaC (FastAPI, AWS SQS, AWS SNS, AWS ECS/EKS, CDK)
- **IGS:** External-customer data ingestion service planning & architecture design (design-only) (Django, Airflow, Kubernetes)
- **SURF Compass:** Device monitoring service planning & design (design-only) (FastAPI, PostgreSQL)
- **RNS:** Automated release-note generation, 50% faster authoring (4h to 2h) (NestJS, Atlassian API)
- **SLT:** 2D LD labeling tool, improved productivity by 30% (C++/MFC)

Korea Institute of Robotics & Technology Convergence, Researcher — Pohang, South Korea — Jan 2022 – July 2022

- Developed real-time gesture-based Human-Robot Interaction system
- Supported autism diagnosis assistance robot project

Skills

Languages: Python, C++, TypeScript
Backend: Django, FastAPI, NestJS
Database: PostgreSQL, MongoDB, Redis
Cloud: AWS (EC2, EKS, ECS, SQS, SNS, Lambda, S3, Bedrock), NCP (NKS, Object Storage, NCR), GCP (Vertex AI)
Infrastructure: Kubernetes (RKE2, NKS), Docker, Terraform, CDK, Rancher
ML/MLOps: Ray, KubeRay, MLflow, KServe, Knative, lakeFS
Workflow: Airflow, Celery
CI/CD: ArgoCD, GitHub Actions, Harbor, Helm
Monitoring: Grafana, Prometheus, Loki, CloudWatch, DCGM, DataDog
Authentication: Keycloak, Okta (SSO integration)
Generative AI / LLM: Local VLM, AWS Bedrock (Claude), Vertex AI (Gemini), multimodal, structured output, RAG / embeddings, prompt engineering, LLM cost & observability, agentic / tool-use, Claude Code / MCP / Skills
Tools: Git, Jira, Confluence, Atlassian API

Generative AI Engineering

- **Tiered multimodal VLM evaluation (production):** Evolved an initial Gemini/Vertex AI service using a 1,110-case golden set: rules resolve 54.7% of a 561-case stratified sample at 96.4% top-1 accuracy, local VLM evaluates the remainder, and ambiguous cases escalate to AWS Bedrock Claude Opus. Production judgments matched validation-team ground truth on 92.6% of issue decisions (718/775, 3 runs). Added self-consistency, JSON-schema output, transfer approval, and human review; unreviewed results are excluded from KPIs (MTBF)
- **LLM cost & usage observability (AWS Bedrock):** Token-level, per-model/per-team cost-per-request attribution via CloudWatch + Slack, Terraform IaC, adopted as an org cost-optimization tool
- **RAG knowledge system (LLM Wiki):** Hybrid keyword + embedding retrieval (text-embedding-3-small, cosine top-k) → grounded answers with citations, freshness, and uncertainty. CI-rebuilt index (Git + Markdown)
- **Agentic LLM tooling:** NL intent → cluster diagnostics/storage tools (Claude Code Skills), multi-step orchestration + guardrails (dry-run, rollback, gates). Multi-model self-critique across Claude/GPT/Gemini (jhwanchoi/dotfiles)
- **Tech:** Local VLM, AWS Bedrock (Claude), Vertex AI (Gemini), embeddings/RAG, Claude Code, Gemini CLI, Codex, MCP, CloudWatch, Terraform, n8n

Education

BS **Handong Global University**, Electronic Engineering & Computer Science — GPA 3.3/4.5 — Feb 2018 – Feb 2022

Coursework: Data Structure, Algorithm, Computer Architecture, Deep Learning, Machine Learning